

实 验 报 告

课程名称	软件测试				
实验项目名称	软件测试试验报告 2				
实验时间 (日期及节次)	第 16 周 3 - 4 节				
专业	软件工程	学生所在学院		计算机与大数据	
年级	2024 级	学号		20245109	
姓名	刘健畅	指导教师		徐为	
实验室名称	4 号楼 - 413 [计算机 1 机房分室 1]				
实验成绩	预习情况	操作技术	实验报告	附加：综合 创新能力	实验 综合成绩
教师签字					

黑龙江大学教务处

一. 实验目的与要求

1. 掌握黑盒测试中的等价类划分法。
2. 能够对输入数据进行分析，合理划分有效等价类和无效等价类。
3. 能够根据等价类表设计精简且覆盖全面的测试用例。
4. 实施测试并记录实际结果，编写规范的测试报告。

二. 实验环境与工具

- 操作系统：Windows 11
- 开发语言及环境：Python, Vscode

三. 被测程序分析

被测程序 VerifyTEL.py 用于验证电话号码的合法性。根据业务逻辑，程序对输入有如下要求：

1. 长度要求：必须恰好为 10 位 或 7 位。
2. 前缀码要求：前缀码首位不能是 0 或 1。
3. 字符要求：电话号码必须全部由数字构成，不能包含字母或其它字符。

四. 等价类划分表

输入条件	有效等价类	编号	无效等价类	编号
号码长度	长度等于 10	1	长度不等于 10 或 7	3
	长度等于 7	2		
前缀码规则	前缀码首位为 2-9	4	前缀码等于 0	5
			前缀码等于 1	6
字符类型	全部由数字组成	7	包含字母	8
			包含特殊字符	9
			包含空格	10
			包含小数	11
			包含汉字	12

五. 测试用例设计与执行结果

用例编号	输入数据	覆盖等价类	预期输出	实际输出	测试结果
01	1234567890	1, 4, 7	123-456-7890 电话号码合法	123-456-7890 电话号码合法	Pass
02	4567890	2, 4, 7	456-7890 电话号码合法	456-7890 电话号码合法	Pass
03	123	3	长度必须为 10 或 7...!	长度必须为 10 或 7...!	Pass
04	0123456	5	前缀码不能以 0 或 1 开头...!	前缀码不能以 0 或 1 开头...!	Pass
05	1234567	6	前缀码不能以 0 或 1 开头...!	前缀码不能以 0 或 1 开头...!	Pass
06	345a678	8	电话号码不合法... 不能出现非数字字符...!	ValueError: invalid literal for int() with base 10: 'a678'	Fail
07	345@678	9	电话号码不合法... 不能出现非数字字符...!	ValueError: invalid literal for int() with base 10: '@678'	Fail
08	345 678	10	电话号码不合法... 不能出现非数字字符...!	345-678 电话号码合法	Fail
09	345.678	11	电话号码不合法... 不能出现非数字字符...!	ValueError: invalid literal for int() with base 10: '.678'	Fail
10	345 嗨 678	12	电话号码不合法... 不能出现非数字字符...!	ValueError: invalid literal for int() with base 10: '嗨 678'	Fail

六. 缺陷分析与测试结论

6.1 缺陷发现与分析

在本次测试执行中，共发现两类严重的健壮性与逻辑缺陷：

1. 异常崩溃缺陷：

当输入恰好为 7 位，但包含字母、特殊符号、小数或汉字时，程序能通过前置的长度和首字符校验。但在执行字符串切片转整型（如 `int(num[3:7])`）时，由于缺乏字符合法性过滤，导致 Python 抛出 `ValueError` 异常，程序直接崩溃。

2. 静默错误缺陷：

当输入包含空格（如 345 678）时，虽然理论上属于非法字符，但由于 Python 的 `int()` 函数会自动忽略字符串前后的空白字符，程序没有拦截报错，反而将其作为合法数据处理并输出了 345-678，存在逻辑漏洞。

6.2 修改缺陷

程序最初的长度校验之后、提取前后缀码之前，增加全局的纯数字校验：

```
if not num.isdigit():  
    print("电话号码不合法...电话号码必须全部由数字组成...\n")  
    return
```

6.3 测试结论

本次黑盒测试运用等价类划分法，共设计并执行了 10 个测试用例。测试结果表明，被测程序对于合法长度和前缀规则的处理逻辑正确，但极其缺乏对异常字符的容错与拦截机制，健壮性较差。应按照 6.2 节的建议增加防御性代码，修复后方可达到标准。

七. 附录

学号：20245109

姓名：刘健畅

```
def VerifyTEL(num):
    if (len(num) != 10 and len(num) != 7):
        print("电话号码不合法...号码长度必须为 10 或 7...!")
        return

    reg = ""
    pre = ""
    suf = ""

    if (len(num) == 10):
        if (num[3] == '0' or num[3] == '1'):
            print("电话号码不合法...前缀码不能以 0 或 1 开头...!")
            return
        reg = int(num[0:3])
        pre = int(num[3:6])
        suf = int(num[6:10])
        print(f"{reg}-{pre}-{suf}\n")
    else:
        if (num[0] == '0' or num[0] == '1'):
            print("电话号码不合法...前缀码不能以 0 或 1 开头...!")
            return
        pre = int(num[0:3])
        suf = int(num[3:7])
        print(f"{pre}-{suf}\n")
    print("电话号码合法\n")

def main():
    VerifyTEL(input("请输入电话号码: "))

if (__name__ == "__main__"):
    main()
```